南京大学

本 科 毕 业 论 文

院　　系　人工智能学院

专　　业　人工智能

题　　目　基于隔离分布核函数的二进制哈希实现高效轨迹检索

年　　级　2020　学　　号　201300068

学生姓名　李鑫澎

指导教师　陈开明　职　　称　教授

提交日期　2024 年 5 月 6 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：基于隔离分布核函数的二进制哈希实现高效轨迹检索

院系：人工智能学院

专业：人工智能

本科生姓名：李鑫澎

指导教师（姓名、职称）：陈开明　教授

摘要：

随着移动设备的普及和物联网技术的快速发展，轨迹数据的产生呈现出爆炸性增长的趋势。轨迹数据包含了移动对象在时空上的轨迹信息，例如车辆出行路径、运动轨迹等，因此在智能交通、移动物体跟踪等众多领域得到了广泛应用。在这些应用中，精确而高效地从轨迹数据库中检索出所需的轨迹是实现任务的关键。然而，传统的轨迹检索方法基于具有二次乃至更高的时间复杂度的轨迹相似度计算方法，无法运用在大规模轨迹数据库上。在此背景下，本文旨在提出一种能够实现高效轨迹检索的哈希方法 VDeH（Voronoi Diagram encoded Hashing）。该方法首先将轨迹数据利用隔离分布核函数（Isolation Distributional Kernel）映射为高维特征空间中的向量，然后通过沃罗诺伊图（Voronoi diagram）将这些向量转化为二进制哈希码，从而实现轨迹数据的压缩存储和后续的高效检索。同时，VDeH 还能够与多种基于哈希的数据检索方法进行结合，进一步提高检索速度。本文通过与现有的哈希学习方法以及经典的轨迹相似度计算方法进行对比实验，验证了所提出方法的有效性和高效性。

关键词：轨迹检索；哈希；隔离分布核函数；机器学习

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Binary Hashing for Efficient Trajectory Retrieval Based on Isolation Distributional Kernel

DEPARTMENT: School of Artificial Intelligence

SPECIALIZATION: Artificial Intelligence

UNDERGRADUATE: Xinpeng Li

MENTOR: Professor Kai Ming Ting

ABSTRACT:

With the popularization of mobile devices and the rapid development of Internet of Things technology, there has been an explosive growth in trajectory data generation. Trajectory data contains information about the spatiotemporal paths of moving objects, such as vehicle travel routes and motion trajectories, and thus has found widespread applications in various fields such as intelligent transportation and mobile object tracking. In these applications, precise and efficient retrieval of desired trajectories from trajectory databases is crucial for the task. However, traditional trajectory retrieval methods based on trajectory similarity calculations with quadratic or even higher time complexity cannot be applied to large-scale trajectory databases. In this context, this paper aims to propose a hashing method called VDeH (Voronoi Diagram encoded Hashing) that achieves efficient trajectory retrieval. This method first maps trajectory data into vectors in a high-dimensional feature space using the Isolation Distributional Kernel, and then transforms these vectors into binary hash codes using Voronoi diagram, thereby enabling trajectory data compression and subsequent efficient retrieval. Additionally, VDeH can be combined with various hashing-based data retrieval methods to further improve retrieval speed. Through comparative experiments with existing learning to hash methods and classical trajectory similarity calculation methods, we validate the effectiveness and efficiency of the proposed approach.

KEYWORDS: Trajectory Retrieval; Hashing; Isolation Distributional Kernel; Machine Learning

目　　录

第一章 绪论 1

第二章 相关工作 3

2.1 轨迹检索与表征 3

2.1.1 轨迹检索 3

2.1.2 轨迹表征 4

2.2 哈希方法 6

2.2.1 哈希检索方法 6

2.2.2 哈希学习 7

2.3 本章小结 9

第三章 基于 IDK 的二进制哈希轨迹检索方法 11

3.1 IDK 表征轨迹 11

3.2 二进制哈希检索 13

第四章 实验测试 17

4.1 实验设置 17

4.2 实验结果 19

4.3 参数分析 24

第五章 总结与展望 27

参考文献 29

致　　谢 35

第一章　绪论

随着移动设备的普及和物联网技术的快速发展，轨迹数据的产生呈现出爆炸性增长的趋势。轨迹数据包含了移动对象在时空上的轨迹信息，例如车辆出行路径、运动轨迹等，因此在智能交通、移动物体跟踪等众多领域得到了广泛应用[1]。在这些应用中，精确而高效地从轨迹数据库中检索出所需的轨迹是实现任务的关键，其中最常见的任务是根据用户查询的轨迹信息检索出最相似的轨迹。一个高效的轨迹检索方法需要能够在大规模轨迹数据库中快速找到与查询轨迹最相似的轨迹，以满足实时应用的需求。

传统轨迹检索方法需要使用动态时间规整（Dynamic Time Warping, DTW）[2]和豪斯多夫距离（Hausdorff distance）[3]等轨迹相似度计算方法。然而，这些传统轨迹相似度计算方法的核心是计算点与点之间的距离，因此具有二次乃至更高的时间复杂度，这导致许多基于这些相似度计算方法的轨迹检索算法无法直接运用在大规模轨迹数据库上[1,4-6]。同时，轨迹的采样率和噪声干扰等因素也使这些基于点对的轨迹相似度计算方法面临诸多挑战[6]。尽管近年来出现了一些利用机器学习方法来快速计算近似轨迹相似度计算的研究[4-5]，但这些方法相比于原本的轨迹相似度计算方法牺牲了一定的精确度，并且在大规模轨迹数据库上的检索速度仍不够理想。

为了解决上述问题，一些研究者开始尝试通过学习轨迹数据的表征（representation），将离散的轨迹数据序列映射到特征空间中，从而能够通过特征空间中向量之间的运算（如欧氏距离）来计算轨迹之间的相似度[6]。隔离分布核函数（Isolation Distributional Kernel, IDK）[7]能够将轨迹数据映射为特征空间中的向量来作为轨迹的表征，现有研究已展现了基于此计算轨迹间相似度的轨迹聚类算法的优越性[8]。

将轨迹表征为向量后，便可以利用已有的数据库检索方法来实现轨迹检索。哈希（hashing），作为一种基于向量的数据表征方法，能够将高维数据映射到低维空间中进行表征，被广泛应用于大规模数据的检索任务中。其中，二进制哈希

（binary hashing）将数据映射到汉明空间（Hamming space）中，由于其极快的距离计算速度而倍受青睐。根据是否依赖数据生成哈希函数，哈希方法主要分为两类：非数据依赖（data-independent）的局部敏感哈希（Locality Sensitive Hashing, LSH）方法和数据依赖（data-dependent）的哈希学习（learning to hash）方法[9]。后者通过机器学习算法来实现数据的哈希，由于其数据依赖的特性，能够更有效地在低维哈希码空间中保留原本数据在高维空间中的相似度，即令学习到的哈希码空间中的最近邻轨迹尽可能接近实际上的最近邻轨迹，因此在近年来得到了广泛的研究。

本研究的贡献在于为应对大规模轨迹数据库，提供一种高效且准确的轨迹检索方法。具体而言，本研究旨在提供一种基于 IDK 和沃罗诺伊图（Voronoi diagram）的二进制哈希技术，将轨迹数据通过 IDK 映射为再生核希尔伯特空间（Reproducing Kernel Hilbert Space, RKHS）中的向量，再利用由沃罗诺伊图得到的二进制哈希映射到汉明空间（Hamming space）作为其表征，通过轨迹表征之间的简单运算便能够获得原本轨迹之间的相似度，从而使得在大规模数据库中快速进行轨迹检索成为可能。这种方法首次使用二进制哈希方法来处理使用 IDK 表征的轨迹数据，不仅有效降低了轨迹数据的存储开销，同时也能实现轨迹数据之间相似度计算的大幅加速，为大规模轨迹数据库的管理和实时应用，如检索最相似轨迹等任务，提供一种快速高效的解决方案。

本文的后续部分组织如下：第二章介绍与本研究相关的工作；第三章详细描述了基于 IDK 的二进制哈希轨迹检索方法 VDeH（Voronoi Diagram encoded Hashing）；第四章通过对比实验验证了 VDeH 方法的有效性和高效性；第五章对本研究的工作进行总结并展望未来的研究方向。

第二章　相关工作

本章将介绍与本研究相关的工作，主要包括轨迹检索与表征、哈希方法两方面的研究进展。

2.1　轨迹检索与表征

2.1.1　轨迹检索

由于轨迹检索在轨迹数据分析任务中的重要性，这个课题在过去数十年来已经得到了广泛的研究[1,6]。传统的轨迹检索算法使用轨迹相似度度量方法来计算查询轨迹与数据库中所有轨迹间的距离，较为著名的轨迹相似度计算方法有动态时间规整[2]，EDR（Edit Distance on Real sequence）距离[10]，豪斯多夫距离[3]和 EDwP（Edit Distance with Projections）距离[11]等等。包括上述方法在内的绝大多数现有度量方法都采用动态规划技术来对不同轨迹的数据点进行对齐，需要计算点与点之间的距离，从而使得计算过程需要至少二次时间复杂度，轨迹检索任务所需的时间开销过大，导致难以在大规模轨迹数据库上得到有效应用[6]。

为了减轻轨迹检索任务的计算开销，许多用于加速检索过程的方法被提出，现有方法主要可以分为两大类。

第一类加速方法利用索引（indexing）技术和剪枝（pruning）技术来降低需要检索的轨迹数量。Chen 等[10]使用 R-树（R-tree）来对轨迹数据建立索引，并结合三种剪枝方法来提升使用 EDR 距离进行相似轨迹检索的速度。Chen 等[12]同样使用 R-树来建立轨迹数据的索引并进行剪枝，以此提高使用 KNN（K-Nearest Neighbors）算法检索最相似轨迹的效率。Xie 等[13]则提出了一种基于片段的划分和索引的分布式框架，将相似度过低的轨迹通过剪枝除去，从而能够在大规模轨迹数据库上进行相似轨迹检索。这些加速方法只是减少了检索轨迹的数量，实际上并没有降低计算两条轨迹数据之间相似度的时间复杂度。因此，它们的应

用场景只局限于 top-k 相似搜索（top-k similarity search）问题，并不能运用在轨迹聚类等需要计算所有轨迹对之间相似度的任务中。

第二类加速方法通过使用近似算法来加速轨迹间相似度的计算，从而达到快速检索轨迹的目标。Yao 等[4]提出的 NeuTraj 模型采样数据集中一定量的轨迹作为“种子轨迹”，并使用度量学习（metric learning）方法基于“种子轨迹”之间的相似度来近似轨迹相似度函数。Yao 等[5]进一步提出了基于图的 TrajGAT 模型以提升在长轨迹上计算轨迹相似度的性能。Indyk 等[14]最早提出使用局部敏感哈希算法来进行轨迹间 Fréchet 距离的近似计算。在此基础上，为了降低其所需的巨大空间开销，Driemel 等[15]以及 Astefanoaei 等[16]则将轨迹数据空间划分为网格，根据轨迹经过的网格序列来构建哈希函数。这类加速方法大幅降低了计算轨迹间相似度的时间开销，从而有效加快了轨迹检索速度，但计算得到的是原本轨迹相似度的近似值，因此只能用于进行近似最近邻搜索（Approximate Nearest Neighbors, ANN）。并且由于这些方法的检索速度和轨迹相似度的近似计算精度仍不够理想，因此在大规模轨迹数据库上的应用仍然受到限制。

2.1.2 轨迹表征

在现实世界中，一条轨迹应当是空间中的一条连续曲线，但由于轨迹数据的采样率等因素，计算机处理的轨迹数据往往是由一系列离散点组成的，传统的基于点对齐的轨迹相似度计算方法在效果和效率上都面临诸多挑战。例如，即使两条轨迹在原本空间上是十分相似的，但由于采样率的不同和噪声的干扰，它们的离散点序列在点对齐的过程中可能会被此类轨迹相似度计算方法认定是不相似的。除此以外，基于点对齐的轨迹相似度计算方法具有二次乃至更高的时间复杂度，需要花费大量的计算资源以找到最佳的点对齐方式，这在大规模轨迹数据库上尤其是长轨迹数据库上是不可接受的。为了解决这些问题，研究者们开始尝试通过学习轨迹数据的表征，对离散点序列形式的轨迹数据进行特征提取，以达到对原本的连续轨迹数据进行建模的目的。这些表征方法将原本复杂的轨迹数据映射为特征空间中的单个向量，同时保留了轨迹数据间的相似度关系，即在特征空间中的向量之间的某种相似度运算结果（如欧氏距离）能够反映向量代表的原本轨迹数据之间的相似度。

Li 等[6]提出了一种基于深度表征学习（deep representation learning）的方法

t2vec，它利用一个基于序列的编码器-解码器（encoder-decoder）模型学习轨迹数据中潜在的信息，从而将一条轨迹数据映射为特征空间中的一个的向量。借助历史轨迹数据和深度神经网络（deep neural network）的表征能力，t2vec 能够学习到轨迹数据中的时空特征，并且对于轨迹数据的噪声和采样率的变化具有一定的鲁棒性。学习到的轨迹表征向量之间利用欧氏距离等简单运算即可用于衡量轨迹之间的相似度，大大降低了轨迹相似度计算的时间开销，同时也能与其他数据检索方法结合以提高轨迹检索速度，或是扩展应用到如轨迹聚类和异常检测等多种轨迹数据分析任务中。

在体育比赛中，常常需要对运动物体的轨迹进行跟踪记录，例如使用相机和传感器在一场足球比赛中将球员和足球的运动轨迹记录下来，供后续的分析和训练使用。相似比赛检索需要在大规模的以往比赛数据库中快速找到与查询比赛最相似的一场或是一个片段，以便进行比赛回顾和制定团队战术，其本质是多条轨迹的检索问题。为了有效和高效地解决相似比赛检索问题，Wang 等[17]提出一种基于深度学习（deep learning）的无监督学习方法 play2vec 来计算两场比赛之间的相似度。play2vec 采用了一种全新的去噪序列编码器-解码器（denoising sequential encoder-decoder）模型，作为已在自然语言处理领域取得成功的负采样跳字（Skip-Gram with Negative Sampling, SGNS）模型[18]的扩展，它能够将比赛数据的片段转化为习得的特征空间中的向量，从而实现比赛数据的表征，进而通过向量之间的欧氏距离来衡量比赛之间的相似度。经过在真实足球比赛数据集上的相似比赛检索实验，play2vec 在准确性，效率和鲁棒性上都有优异的表现。

Wang 等[8]提出使用隔离分布核函数（Isolation Distributional Kernel, IDK）[7]来表征轨迹数据并用于在轨迹聚类等问题中计算轨迹间相似度。这种方法将原本空间中连续的轨迹视作一个未知的概率分布，而离散的轨迹序列数据则是从其中独立同分布（independent and identically distributed, i.i.d.）采样得到的样本。已有的轨迹相似度计算方法，如推土机距离（Earth Mover’s Distance, EMD，也称为 Wasserstein 距离），同样将轨迹数据视作概率分布，但它需要使用直方图来近似建模轨迹数据的概率分布，这使得其在计算轨迹相似度时精度无法得到保障且具有很高的时间复杂度。相比之下，使用 IDK 等分布核函数（distributional kernel）来表征轨迹数据并不需要估计原本轨迹的概率分布，而是直接利用从分布中采样得到的样本点，通过核均值嵌入（kernel mean embedding）[19]将点对间

（point-to-point）的核函数推广到分布间的核函数，从而能够通过分布核函数对应的特征映射将轨迹数据作为一个分布映射为再生核希尔伯特空间（Reproducing Kernel Hilbert Space, RKHS）中的特征向量。经过实验验证，利用 IDK 表征的轨迹数据在轨迹聚类任务中取得了优异的性能，同时在检索相似轨迹的任务中也展现出了超越传统轨迹相似度计算方法以及诸多近年来新提出方法的优势，表明 IDK 能够有效地表征轨迹数据并用于计算轨迹间相似度。

2.2 哈希方法

哈希（hashing）能够将高维复杂数据映射到低维空间中以一条哈希码进行表征，被广泛应用于大规模数据检索、近似最近邻搜索等任务中。

2.2.1 哈希检索方法

与加速轨迹检索过程的方法类似，哈希方法根据其利用哈希码进行检索的方式也可以分为两类：哈希表查找（hash table lookup）和哈希码排序（hash code ranking）[9]。

哈希表查找方法利用哈希函数构建哈希表（hash table），一个哈希表由多个哈希桶（hash bucket）组成，每个哈希桶对应着一个独特的哈希码。数据库中的数据已提前通过哈希函数映射到了哈希表中的某个哈希桶中，当需要检索某个数据时，只需考虑与其具有相同哈希码的数据，即只需在其哈希码对应的哈希桶中查找即可，从而大幅减少了需要检索的数据量。与传统的哈希方法试图减少数据在哈希表中的冲突（collision）不同，哈希表查找方法并不要求避免将不同数据映射到同一个哈希桶中，反而是利用这种冲突来让相似的数据能够映射到相同的哈希桶中，而不相似的数据则尽可能不被映射到同一个哈希桶中，这实际上是利用哈希码来近似数据之间的相似度。对于同一个哈希桶中的数据，往往需要根据原本的数据相似度计算方法进一步计算它们之间的相似度，这一过程的效率取决于哈希表中每个哈希桶中的数据量和相似度计算方法的时间复杂度。哈希表查找方法通常与局部敏感哈希结合使用，但由于其只是减少了检索数据量而并没有减少相似度计算的时间复杂度，因此在大规模数据检索任务中的效率仍不够理想，在现实中的应用也较为有限[20]，但基于量化（quantization）哈希

码的哈希表查找方法在粗略获取潜在近似最近邻数据上有着不错的表现[21]。为了提高召回率（recall），实际应用中常常会搜索多个哈希桶或是构建多个哈希表，牺牲一定的检索效率以换取更好的检索效果。

哈希码排序方法则是将数据映射为哈希码后，根据哈希码之间的距离来衡量数据之间的相似度，从而实现数据的排序。例如对于二进制哈希码，可以使用汉明距离（Hamming distance）来衡量两个哈希码之间的距离，由于汉明距离可以使用 CPU 指令级的位运算（如 POPCNT）来计算，具有极快的计算速度。哈希码排序方法极大地降低了数据之间相似度的计算时间开销，使得在大规模数据检索任务中能够快速找到与查询数据最相似的数据，但它的检索性能取决于能否在哈希码空间中最大限度地保留数据在原本空间中的相似度，并且需要遍历所有数据以计算其哈希码之间的距离，这可以通过与上文的哈希表查找方法结合使用来进一步提高检索效率。对于聚类、异常检测等任务，由于哈希码排序方法直接降低了数据之间相似度的计算时间开销，因此在这些任务中也能得到广泛的应用。

2.2.2 哈希学习

传统的局部敏感哈希（Locality Sensitive Hashing, LSH）方法[22]基于随机投影（random projection），需要提前针对近似的相似度计算方法设计好哈希函数，并不考虑数据本身的性质，这使得其在处理不同分布的数据时效果不尽如人意[9]。为了解决这一问题，研究者们开始尝试通过机器学习方法来从给定数据学习得到哈希函数，即哈希学习（learning to hash）方法。学习到的哈希函数相比于非数据依赖的局部敏感哈希方法，能够使得数据在哈希码空间中的表征更好地保留数据在原本空间中的相似度，即哈希码越相近的数据之间原本的相似度应当也越高，同时得到的哈希码还具备较低的检索时间和空间开销。由于数据依赖性带来的优势，哈希学习已经在如大规模图像分类[23]和数据检索[24]等领域得到了广泛的应用。下面介绍几种将在后续章节中用于比较的哈希学习方法。

Gong 等[25]提出的迭代量化（Iterative Quantization, ITQ）是一种经典的哈希学习方法，它把原始数据通过主成分分析（Principal Component Analysis, PCA）等降维方法进行降维投影后，将学习哈希码的问题转化为学习一个二值化矩阵，使得数据被映射到一个二进制超立方体（binary hypercube）的顶点上，并最小化

该过程对应的量化误差（quantization error）。对于主成分分析降维后的零均值化（zero-centered）数据，ITQ 通过迭代优化的方式学习一种最优的旋转（rotation）矩阵，即正交转换（orthogonal transformation）矩阵，使得数据在经过旋转后，被映射到二进制超立方体的顶点上时产生的量化误差最小，这种方法相比过去实际常用的随机正交转换矩阵[24]能够得到更好的哈希码质量。ITQ 还可以与典型相关分析（Canonical Correlation Analysis, CCA）[26]结合以使用数据标签进行监督学习（supervised learning），或是与核方法（kernel method）结合以处理非线性数据。但由于 ITQ 基于降维后的数据进行学习正交转换矩阵，因此无法利用比原始维度更多的二进制比特位来表征数据，这使得其在表征能力上受到了一定限制。

除了显式地通过降低原本数据相似度与哈希码表征相似度之间的差异来学习哈希函数外，还有一些方法通过对空间进行划分来构建哈希函数。现有的许多此类哈希函数使用超平面（hyperplane）对数据进行划分并分配哈希码，而 Heo 等[27]提出的球哈希（Spherical Hashing, SpH）通过构造基于超球体（hypersphere）的哈希函数来实现数据的哈希化，以一种球汉明距离（spherical Hamming distance）作为数据之间的距离度量，并通过迭代优化的方式取得最优的哈希码。与使用超平面对数据空间进行划分相比，超球体从空间划分出封闭区域（closed region）的能力更强。例如，使用超平面划分一个 d 维空间时，需要使用至少 $d+1$ 个超平面才能划分出一个封闭区域，而使用超球体则只需要一个超球体即可划分出一个封闭区域，而使用与超平面相同数量的超球体能够划分出更多的封闭区域。与多个超平面划分出的封闭区域相比，超球体划分出的封闭区域内数据点之间的最大距离受到了限制，这对最近邻搜索等任务中快速检索数据点的效率有着积极的影响。SpH 采用的球汉明距离比传统的汉明距离更适合用于度量其产生的哈希码之间的距离，同时 SpH 还使用迭代优化以取得数据的均衡划分以及哈希函数之间的独立性，从而能够生成更好的哈希码。SpH 迭代优化的过程在考虑所有数据时需要较大的时间开销，在大规模数据集上需要通过对数据采样等方式来减少计算量以保证其训练效率。

有研究者指出简单的基于随机投影的 LSH 方法长久以来遭到了低估[28]，但其性能的优越性只有在哈希码长度足够长，即使用的随机投影数量足够多时才能体现。使用过长的哈希码会导致计算和存储开销的增加，从而使得 LSH 方法在大规模数据集上的实际应用受到限制。此外，尽管 LSH 方法的哈希函数是随

机独立生成的，但其哈希函数之间仍可能存在一定的相关性，例如两个哈希函数使用了同一个随机投影矩阵。尤其在哈希码长度较长时，LSH 方法使用的哈希函数会有较高的相关性和冗余性，从而影响哈希码的表征能力。为了解决上述问题，Liu 等[29]改进了原本非数据依赖的 LSH 方法，提出了数据依赖的局部敏感哈希的比特位缩减方法（bit reduction for LSH, brLSH），并进一步提出了精炼局部敏感哈希二进制码的方法（refining binary codes for LSH, rcLSH），使得其相比于传统基于随机投影的 LSH 方法只需要少量高质量的哈希函数即可取得更好的哈希码。一个表征能力强的哈希函数应当能够将数据划为较均等的两部分，即 0 和 1 的比特位数量大致相同，同时哈希函数的划分需要符合数据间的相似性，相似的数据应当被分配到相同的比特值，而不相似的数据则应当分配不同的比特值。brLSH 结合这两方面对 LSH 生成的随机投影进行评估，衡量随机产生的哈希函数的优劣，并取最优的一定数量的哈希函数作为最终使用的哈希函数，从而大大减少了哈希码的长度。作者通过实验测试发现，虽然 brLSH 并没有进行显式的学习，但由于其数据依赖性，即使使用更短的哈希码长度，brLSH 在数据检索任务中的表现仍然优于未进行比特位缩减的传统 LSH 方法。在此基础上，rcLSH 进一步通过迭代优化的方式对哈希码进行精炼，降低哈希码中的冗余信息以及比特位之间的相关性，使得生成哈希码的表征能力得到了进一步提升。但 rcLSH 的迭代优化过程需要使用到完整的 $n \times n$ 拉普拉斯相似度矩阵（Laplacian affinity matrix）来衡量 n 个数据两两之间的相似性，这在大规模数据集上会带来极大的时间和空间开销，因此无法直接应用于大规模数据检索任务中。

2.3 本章小结

本章介绍了与本研究相关的背景和现有方法。用于加速轨迹检索过程的方法主要可以分为两类：基于索引和剪枝技术的加速方法以及基于近似算法的加速方法，前者减少了数据库中需要检索的轨迹数量，但并没有降低计算两条轨迹数据之间相似度的时间复杂度，而后者通过以牺牲一定的精度来换取更快的近似相似度计算速度，但其在大规模轨迹数据库上的效率仍不够理想。为了进一步减轻轨迹间相似度的计算开销，以及达到对原本空间中的连续轨迹进行建模的目的，研究者们开始尝试通过学习轨迹数据的表征，将离散点序列形式的轨迹数

据映射为特征空间中的单个向量，从而能够通过向量之间的某种相似度运算结果来衡量轨迹之间的相似度。将轨迹数据映射为特征空间中的向量后，便可以利用数据检索任务中常用的哈希方法将其映射为低维空间中的哈希码，以此来加速轨迹数据的检索。哈希方法主要包括两类：哈希表查找方法和哈希码排序方法。哈希表查找方法通过构造哈希表来将数据分配到不同的哈希桶中，减少需要检索的数据量，而哈希码排序方法则是通过快速计算哈希码之间的距离来衡量数据之间的相似度，从而实现数据的排序。前者的优缺点与基于索引和剪枝技术的轨迹检索加速方法类似，而后者虽然运算速度极快，但检索性能参差不齐且需要遍历数据库中的所有数据以计算其哈希码之间的距离，可以通过与哈希表查找方法结合使用来进一步提高检索效率。在哈希方法中，哈希学习方法通过机器学习的方式来学习哈希函数，其数据依赖性使得数据在哈希码空间中的表征能够更好地保留数据在原本空间中的相似度。与非数据依赖的哈希方法 LSH 相比，哈希学习方法更加复杂，需要花费时间提前从数据库中进行学习，但其生成的哈希码质量更好，能够有效提高数据检索的性能，因此在近年来得到了广泛的研究和应用。

第三章　基于 IDK 的二进制哈希轨迹检索方法

3.1　IDK 表征轨迹

核方法（kernel method）在机器学习领域被广泛应用于数据表征、分类和模式识别等任务中。每个核函数 κ 都隐式地唯一对应着一个特征映射 ϕ，它能将原始空间中的数据点 $\mathbf{x} \in \mathbb{R}^n$ 映射为高维特征空间再生核希尔伯特空间（Reproducing Kernel Hilbert Space, RKHS）$\mathcal{H}$ 中的特征向量 $\phi(\mathbf{x}) \in \mathcal{H}$[30]。特征向量之间的内积 $\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle_{\mathcal{H}}$ 等价于核函数 $\kappa(\mathbf{x}, \mathbf{y})$ 的计算结果，点对间（point-to-point）的核函数值衡量了数据点之间的相似度。常用的核函数包括

- 线性核（Linear Kernel）：$\kappa(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$;
- 高斯核（Gaussian Kernel）：$\kappa(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2}\right)$;
- 拉普拉斯核（Laplacian Kernel）：$\kappa(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}\right)$.

隔离核函数（Isolation Kernel, IK）[31]是一种新型的点对间核函数，与高斯核函数等传统核函数相比，IK 具有两个使其更适合用于表征数据的优势：

1. IK 对应的特征映射是有限维的，而高斯核函数和拉普拉斯核函数等传统核函数对应的特征映射是无限维的。我们显然无法使用无穷的维度对数据进行表征，尽管可以利用例如 Nyström 方法[32]或是随机傅里叶特征（random Fourier features）[33]等技术来近似得到有限维的特征映射，但这些方法往往会导致性能的下降并伴随着较大的计算开销。
2. IK 具有数据依赖性，即 IK 的特征映射是根据数据点的分布而生成的，而高斯核函数等非数据依赖的传统核函数不考虑数据的分布。数据依赖性让 IK 能够更好地利用数据的分布信息，使得具有同样距离的两点在稀疏区域的相似度比在密集区域更高，这令 IK 更适合用于表征数据。

实现 IK 的其中一种方法是以 iNNE（isolation using Nearest Neighbor Ensemble）[34]的方式对空间进行随机划分。具体而言，记数据集为 $\mathcal{D} \subset \mathbb{R}^n$，从中随机

采样 ψ 个数据点 $\{\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_\psi\} \subset \mathcal{D}$ 作为一个划分，其中每个点 $\mathbf{z}_i$ 由一个以其自身为中心的超球体 θ_i 与其余 $\psi - 1$ 个点隔离开，θ_i 的半径 r_i 即为 $\mathbf{z}_i$ 到剩下 $\psi - 1$ 个点中的最近邻点的距离。通过这种方式，$\mathcal{D}$ 中的每个点都属于一个超球体，或是处于所有超球体之外（若一个点恰好位于多个超球体内，则将其分配到离其中心点最近的超球体内）。对于第 j 次划分，可以构建如下定义的 ψ 维二元列向量 $\phi_j(\mathbf{x}|\mathcal{D})$：

$$\phi_j(\mathbf{x}|\mathcal{D}) = \begin{cases} \mathbf{0} \quad \text{if} \quad d_{\min} > \max\limits_{i \in [\psi]} r_i \\ [0, \cdots, 0, 1(k\text{th position}), 0, \cdots, 0]^\top \quad \text{otherwise} \end{cases} \tag{3-1}$$

其中 $d_{\min}$ 为 $\mathbf{x}$ 到 $\{\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_\psi\}$ 中的最近邻点 $\mathbf{z}_k$ 的距离。

此后，在不产生歧义的情况下，我们省略 $|\mathcal{D}$，将 $\phi_j(\mathbf{x}|\mathcal{D})$ 简记为 $\phi_j(\mathbf{x})$。

重复上述划分构建过程 t 次，得到 t 个 ψ 维二元列向量 $\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \cdots, \phi_t(\mathbf{x})$，将其拼接后即可得到数据点 $\mathbf{x} \in \mathcal{D}$ 的完整 IK 特征映射 $\Phi(\mathbf{x}) : \mathbb{R}^n \to \{0, 1\}^{t \times \psi}$：

$$\Phi(\mathbf{x}) = [\phi_1(\mathbf{x})^\top, \phi_2(\mathbf{x})^\top, \cdots, \phi_t(\mathbf{x})^\top]^\top. \tag{3-2}$$

作为一个点对间核函数，IK 能够利用数据集 $\mathcal{D}$，通过以上特征映射得到的 $t \times \psi$ 维特征向量之间的内积来衡量两点之间的相似度，即

$$\kappa(\mathbf{x}, \mathbf{y}) = \langle \Phi(\mathbf{x}), \Phi(\mathbf{y}) \rangle \tag{3-3}$$

在 IK 的基础上，隔离分布核函数（Isolation Distributional Kernel, IDK）[7]被用于衡量两个概率分布之间的相似度。使用核均值嵌入（kernel mean embedding）[19]能够将点对间核函数 IK 的特征映射推广到分布间核函数 IDK 的特征映射，从而使得 IDK 能够将数据集 $\mathcal{S} \subset \mathbb{R}^n$ 对应的潜在分布 $\mathcal{P}_\mathcal{S}$ 映射为 RKHS 中的一个特征向量 $\widehat{\Phi}(\mathcal{P}_\mathcal{S}) \in \mathcal{H}$：

$$\widehat{\Phi}(\mathcal{P}_\mathcal{S}) = \frac{1}{|\mathcal{S}|} \sum_{\mathbf{x} \in \mathcal{S}} \Phi(\mathbf{x}) \tag{3-4}$$

那么两个分布 $\mathcal{P}_{\mathcal{S}}$ 和 $\mathcal{P}_{\mathcal{T}}$ 之间 IDK 值的经验估计即为

$$\begin{aligned}\mathcal{K}(\mathcal{P}_{\mathcal{S}}, \mathcal{P}_{\mathcal{T}}) &= \frac{1}{|\mathcal{S}||\mathcal{T}|} \sum_{\mathbf{x}\in\mathcal{S},\mathbf{y}\in\mathcal{T}} \langle \Phi(\mathbf{x}), \Phi(\mathbf{y}) \rangle \\ &= \langle \widehat{\Phi}(\mathcal{P}_{\mathcal{S}}), \widehat{\Phi}(\mathcal{P}_{\mathcal{T}}) \rangle\end{aligned} \tag{3-5}$$

作为由 IK 衍生出的分布核函数，IDK 自然而然地继承了 IK 的优点，即 IDK 对应的特征映射是有限维的，且具有数据依赖性。

轨迹数据作为离散的坐标点序列，通过对原本空间里的连续轨迹进行采样得到，可以被视作从一个未知的概率分布中独立同分布采样得到的样本。现有的轨迹相似度计算方法如推土机距离（Earth Mover's Distance, EMD）同样将轨迹视作概率分布，但相比之下，IDK 具有以下三点优势：

1. IDK 不需要估计原本轨迹的概率分布，而是直接利用从分布中采样得到的样本点计算轨迹数据间的相似度，而 EMD 需要对原本轨迹的分布进行建模，这个过程会引入一定的误差。
2. IDK 的特征映射具有线性时间复杂度，而 EMD 具有至少二次的时间复杂度，这使得 IDK 在计算轨迹相似度时具有更高的效率。
3. IDK 的特征映射是单射的，这保证了当且仅当两个分布完全相同时，它们的特征映射才相等，即 $\|\widehat{\Phi}(\mathcal{P}_{\mathcal{S}}) - \widehat{\Phi}(\mathcal{P}_{\mathcal{T}})\|_{\mathcal{H}} = 0$ 当且仅当 $\mathcal{P}_{\mathcal{S}} = \mathcal{P}_{\mathcal{T}}$。这是因为 IK 是一个特征核（characteristic kernel）[7,19]。EMD 以及现有的表征学习方法都无法保证这一性质。

因此，我们使用 IDK 来有效地表征轨迹数据，已有研究证明这种做法在轨迹聚类等任务中有着优异的表现[8]。

3.2 二进制哈希检索

将轨迹数据通过 IDK 映射为 RKHS 中的特征向量后，我们可以利用一种基于沃罗诺伊图（Voronoi diagram）的哈希方法将其进一步转化为二进制哈希码，以便在大规模数据集上进行快速检索。

沃罗诺伊图是一种空间分割方法，它将空间中的点集以某种方式划分为若干个不重叠的区域。在最常见的情况下，我们可以利用空间中的点集 $\{\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_n\}$

作为种子点集构建一个沃罗诺伊图，其中每个种子点 $\mathbf{z}_i$ 对应着一个沃罗诺伊原胞（Voronoi cell）V_i，即空间中所有最近邻为 $\mathbf{z}_i$ 的点构成的区域。图3-1展示了以欧氏距离作为距离度量的沃罗诺伊图在二维空间和三维空间中的例子[35]。

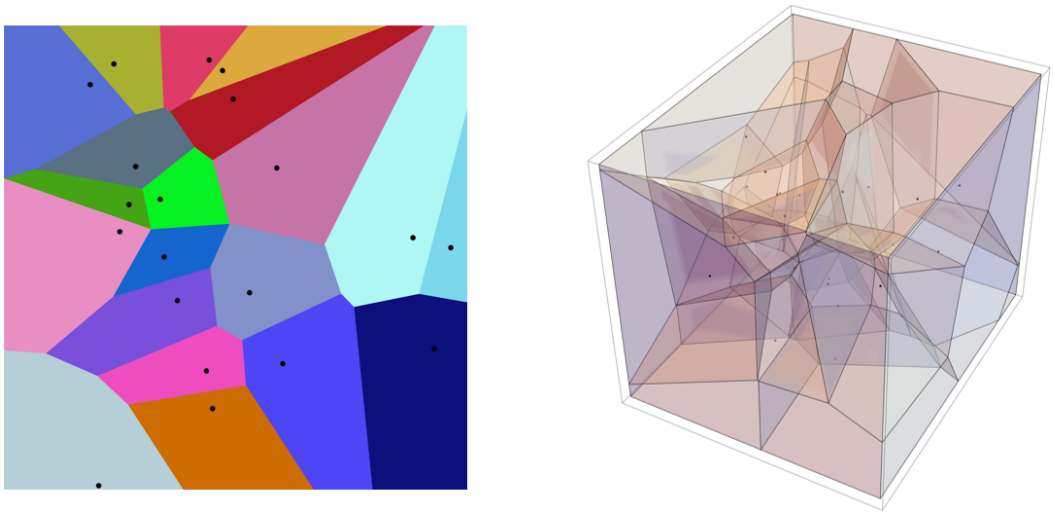


图 3-1　二维空间和三维空间中的沃罗诺伊图

对于 RKHS 中表征轨迹数据 T 的特征向量 $\hat{\Phi}(\mathcal{P}_T) \in \mathcal{H}$，我们可以通过计算其与每个种子点间的欧式距离来确定其所属的沃罗诺伊原胞 V_k，即 T 在此沃罗诺伊图中的标签为 k。具有相同标签的特征向量在 RKHS 中距离较近，并且由于 IDK 能够保留数据在原本空间中的相似度，在 RKHS 中距离相近的特征向量对应的原始轨迹数据在原本空间中应当也具有较高的相似度。因此，$\hat{\Phi}(\mathcal{P}_T)$ 在沃罗诺伊图中的标签 k 可以作为 T 的二进制哈希码的一部分，即 T 的二进制哈希码片段的第 k 位为 1，其余位为 0。通过多次构建沃罗诺伊图并将其结果拼接，我们便能将轨迹数据的特征向量转化为的一段指定长度的完整二进制哈希码。计算两个轨迹数据的二进制哈希码之间的汉明距离即可衡量它们之间的相似度，从而实现轨迹数据的快速检索。我们将这种基于 IDK 和沃罗诺伊图的二进制哈希轨迹检索方法的哈希方法称为 Voronoi Diagram encoded Hashing（VDeH）。

对于数据检索任务，基于哈希码排序的方法需要在整个数据库中进行搜索，这意味着即使是与查询数据十分不相似的数据也需要被考虑，导致大量的时间被消耗在与不相似数据之间的相似度计算上。如果能够在检索之前通过某种方式快速筛选出与查询数据相似的数据，提前排除掉大量的较不相似数据，就能够大幅提高检索速度。

受此启发，我们在本研究中进一步实现了 VDeH 与分组汉明排序（Grouped Hamming Ranking, GHR）[28]方法的结合，以提高 VDeH 在大规模数据集上的检索效率。对于轨迹数据集经过 IDK 特征映射表征得到的特征向量 $\{\mathbf{v}_1, \mathbf{v}_2, \cdots, \mathbf{v}_n\}$ 和查询轨迹数据的 IDK 特征向量 $\mathbf{q}$，分组汉明排序方法的步骤如下：

1. 使用 k 均值聚类（k-means clustering）算法将所有特征向量分为 G 个簇 $\mathcal{C}_1, \mathcal{C}_2, \cdots, \mathcal{C}_G$，其中第 i 个簇 $\mathcal{C}_i$ 的质心（centroid）为 $\mathbf{c}_i$。
2. 计算 $\mathbf{q}$ 与每个簇质心 $\mathbf{c}_i$ 之间的欧式距离，找到与 $\mathbf{q}$ 距离最近的 g 个簇 $\mathcal{C}_{i_1}, \mathcal{C}_{i_2}, \cdots, \mathcal{C}_{i_g}$。
3. 将 $\mathbf{q}$ 通过 VDeH 转换为二进制哈希码 $\mathbf{b}$，并计算 $\mathbf{b}$ 与属于这 g 个簇的所有二进制哈希码（即 $\mathcal{C}_{i_1}, \mathcal{C}_{i_2}, \cdots, \mathcal{C}_{i_g}$ 中的所有特征向量通过 VDeH 转换得到的二进制哈希码）之间的汉明距离。
4. 对于 $\mathcal{C}_{i_1}, \mathcal{C}_{i_2}, \cdots, \mathcal{C}_{i_g}$ 中的轨迹特征向量，返回其中对应汉明距离最小的 L 个作为最终的检索结果。

其中参数 g 和 L 可以根据实际需求进行调整，以平衡检索速度和检索效果。

在 VDeH 直接降低了轨迹数据间相似度计算时间开销的基础上，GHR 通过提前对数据库进行分组来避免计算查询数据与数据库中所有数据之间的相似度，从而进一步提高了检索效率。

第四章　实验测试

在本章中，我们将通过多个数据集上的实验测试来验证 VDeH 的有效性。

4.1　实验设置

我们将 VDeH 与以下几种方法进行比较：

- 迭代量化（Iterative Quantization, ITQ）[25]：ITQ 通过迭代优化的方式学习一个最优的旋转矩阵，将数据映射到二进制超立方体的顶点上，以最小化量化误差。
- 球哈希（Spherical Hashing, SpH）[27]：SpH 使用基于超球体的哈希函数将数据转化为二进制哈希码。在本次实验中，SpH 随机选取数据库中的十万条数据作为训练集以降低迭代优化的时间开销。
- 比特位缩减局部敏感哈希（bit reduction for LSH, brLSH）[29]：brLSH 通过评估 LSH 随机产生的哈希函数的优劣，取最优的的部分哈希函数使用，从而减少哈希码的长度。在本次实验中，我们让 LSH 生成四倍于 brLSH 的哈希函数数量，并随机选取数据库中的一万条数据作为训练集以降低时间和空间开销。
- 豪斯多夫距离（Hausdorff distance）[3]：一种常用的轨迹间距离计算方法，两条轨迹之间的豪斯多夫距离定义为 $H(T_1, T_2) = \max\{h(T_1, T_2), h(T_2, T_1)\}$，其中 $h(T_1, T_2) = \max_{\mathbf{x} \in T_1} \min_{\mathbf{y} \in T_2} \|\mathbf{x} - \mathbf{y}\|$。

关于这些方法的更多细节已在2.2.2节中介绍。

实验在以下三个真实数据集上进行：

- T-Drive 数据集[36]：记录了北京市 33000 辆出租车在 3 个月内的 GPS 坐标数据。
- Porto 数据集[37]：包含了葡萄牙波尔图市从 2013 年 7 月至 2014 年 6 月之间的出租车轨迹数据。

- NYC 数据集[38]：记录了纽约市四年间的出租车轨迹数据，包括订单起止时间地点、计费距离等信息。

我们对这些原始数据进行预处理后得到用于实验的轨迹数据集。具体而言，我们将每辆出租车的坐标数据按时间顺序连接起来，坐标点之间的时间间隔小于 1 小时则被视作同一段轨迹，同时过滤掉长度小于 10 的轨迹。对于得到的轨迹数据集，我们随机选取其中的 1000 条轨迹作为查询数据，其余轨迹作为数据库数据。数据库中的轨迹用于构建 IDK 特征映射和 VDeH 沃罗诺伊图，其 IDK 特征向量用于训练哈希学习方法，并已经提前转换为二进制哈希码。由于原始数据并不携带基准真实值（ground truth），所有方法以轨迹的 IDK 特征向量之间的欧氏距离作为基准真实值。关于数据集的详细信息总结在表4-1中。

表 4-1　数据集详细信息

数据集	T-Drive	Porto	NYC
轨迹数量	745,804	1,641,032	11,926,596
点数量	16,023,166	83,210,527	277,005,055
轨迹长度范围	10 - 3,438	10 - 3,881	10 - 244
平均轨迹长度	21.5	50.7	23.2
查询轨迹数量	1,000		
数据库轨迹数量	744,804	1,640,032	11,925,596
IDK 特征向量维数	1,600		

我们以 1000 次查询的平均精度均值（mean average precision, mAP）[39]作为评价指标，对比 VDeH 与其他方法在三个数据集上的检索效果，这种 mAP 的实现方式已经在许多现有工作中得到采用[40-41]。在 T-Drive/Porto/NYC 数据集上，mAP 分别以查询数据与其第 100/500/1000 真实最近邻 IDK 特征向量之间的欧氏距离的平均值作为阈值，判断一个检索结果是否为真正例（true positive）。此外，为了验证 GHR 方法对 VDeH 的检索效率提升，我们在 1000 最近邻搜索任务上探究了召回率（recall）与检索结果数量和检索时间之间的关系。GHR 在 T-Drive/Porto/NYC 数据集上分别将 IDK 特征向量通过 k 均值聚类算法分为 $G =$ 74/164/1192 个簇，并取 $g = 3/6/30$ 个最近簇中的数据作为候选最近邻。

实验在一台配备 AMD EPYC 7742 处理器的计算机上进行。我们使用 O2 优化级别编译的 C++ 代码单线程实现数据检索过程，其余所有方法均使用 MATLAB 实现，并使用不超过 64 线程进行运算。

4.2 实验结果

图4-1至图4-3展示了 VDeH 与其他方法在三个数据集上的性能表现，其中横轴为二进制哈希码长度，纵轴为 mAP 值。

从图4-1中可以看出，在 T-Drive 数据集上，当二进制哈希码长度低于 512 位时，VDeH 与其他方法相比并没有明显优势，但随着哈希码长度的增加，VDeH 的性能逐渐提高，并最终在哈希码长度超过 1024 位之后成为表现最好的方法。而通过图4-2和图4-3我们可以看到，在 Porto 数据集和 NYC 数据集上，VDeH 的表现更加出色，从哈希码长度为 64 位的一开始就超过了其他所有方法，并且其性能随着哈希码长度的增加还在不断提升。这是因为更长的二进制哈希码意味着 VDeH 构建了更多的沃罗诺伊图，从而能够在哈希码空间中更好地保留数据在 RKHS 中的相似度，提高检索效果。

观察其他方法的表现可以发现，ITQ 仅在 T-Drive 数据集上且使用中长二进制哈希码长度时表现较好，在另外两个数据集上或是哈希码长度较短时表现十分糟糕。并且由于基于 PCA 降维的 ITQ 方法无法利用比原始维度更多的二进制比特位来表征数据，其在哈希码长度为 2048 位时实际上只使用了 1600 位（与 IDK 特征向量维数相同）。SpH 只在 T-Drive 数据集上且二进制哈希码长度较短时表现比较好，使用更多的哈希码位数并没有带来性能的明显提升。事实上，由

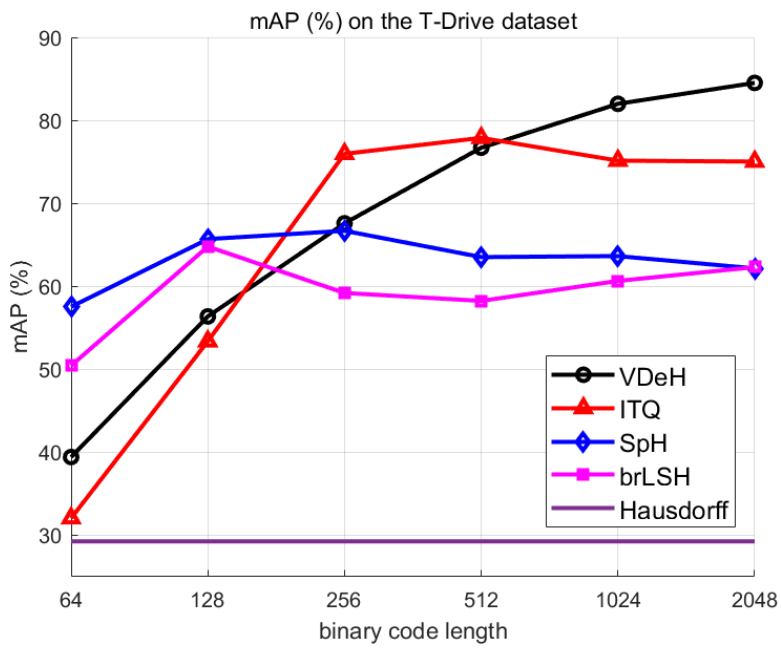


图 4-1　T-Drive 数据集上的 mAP（%）

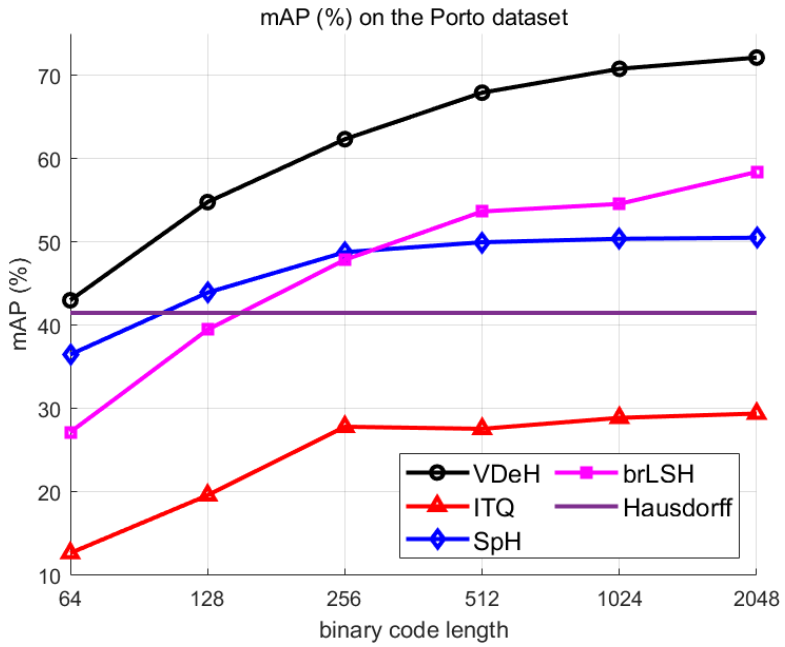


图 4-2　Porto 数据集上的 mAP（%）

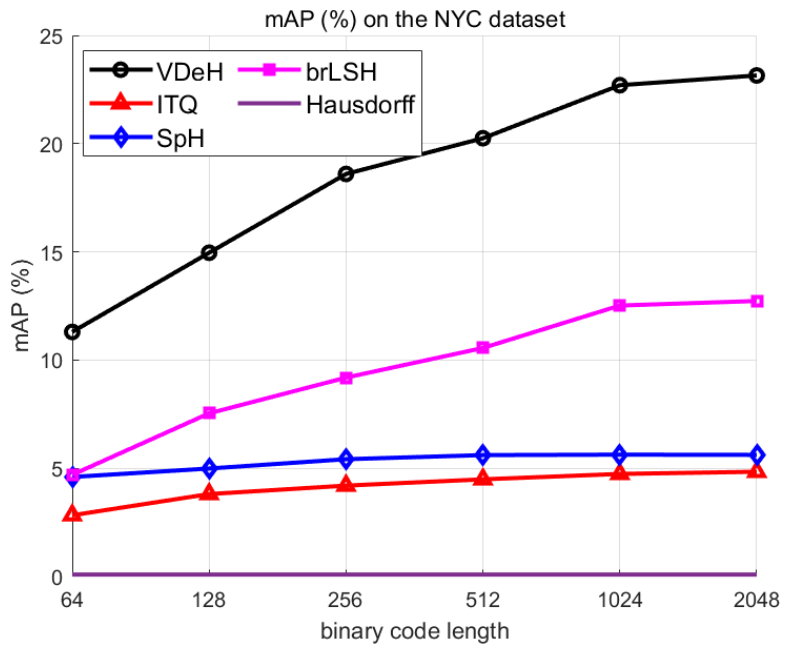


图 4-3　NYC 数据集上的 mAP（%）

于 ITQ 和 SpH 都基于优化方法，其最佳性能往往只在取某个特定的二进制哈希码长度时才能体现，这与 VDeH 随着哈希码长度的增加而逐渐提升的性能表现有所不同。brLSH 在 Porto 和 NYC 数据集上的性能不如 VDeH，并随着二进制哈希码长度的增加而提升，这与预期相符，但其在 T-Drive 数据集上出现了哈希码长度增加时性能反而下降的情况，这可能是因为 LSH 产生的随机哈希函数质量不够稳定，导致 brLSH 在选择最优哈希函数时出现了问题。由于豪斯多夫距离并不是一种哈希方法，其性能不会随着二进制哈希码长度的不同而有所变化，在此处作为轨迹相似度的另一种计算方法展示其与基准真实值之间的差异。

表4-2展示了各方法在三个数据集上进行 1000 最近邻搜索需要的查询时间（查询 10 次取平均），包括以 VDeH 为代表的二进制哈希方法在不同哈希码长度下的查询时间（包含将查询数据转换为二进制哈希码的时间），使用原始轨迹数据进行轨迹间相似度计算的豪斯多夫距离的查询时间，以及直接计算基准真实值（轨迹 IDK 特征向量之间的欧氏距离）进行查询所需的时间。可以看到，使用豪斯多夫距离或是使用基准真实值精确检索最近邻轨迹需要的时间可达几十乃至几百秒，显然无法满足大规模数据集上的实时检索需求。而使用 VDeH 等二进制哈希方法进行检索时，即使在千万级的 NYC 数据集上使用最长的 2048 位二进制哈希码，所需的查询时间也保持在毫秒级别，相比之下检索速度有着极大的优势。

表 4-2　查询时间

数据集		T-Drive	Porto	NYC
二进制哈希	64 位	1.42ms	3.27ms	25.08ms
	512 位	7.69ms	18.11ms	146.56ms
	2048 位	32.24ms	75.43ms	537.58ms
豪斯多夫距离		40.74s	191.55s	714.16s
基准真实值		6.32s	12.41s	90.86s

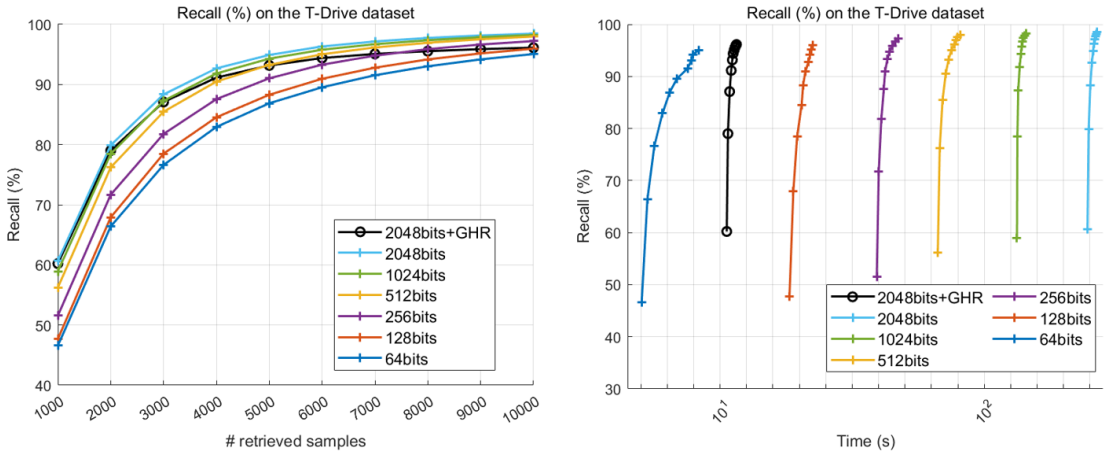
在进行检索前，哈希方法需要基于数据库中的数据学习哈希函数并将这些数据转换为二进制哈希码，这一过程需要花费一定的时间。表4-3展示了当哈希码长度为 1024 位时，VDeH 等四种哈希方法在 T-Drive 数据集上的学习时间和数据转换时间，以及总时间。从表中可以看出，VDeH 在学习时间上与其他方法相比有着很大的优势，这是因为 VDeH 并不需要进行显式的学习，其具有数据依赖性的学习实际上是沃罗诺伊图的构建，这一过程的时间开销非常小。另一方面，VDeH 将更多的时间花费在通过沃罗诺伊图将数据转换为二进制哈希码的过

程中。相比之下，其它方法的时间开销主要在于学习过程，而数据转换时间则相对较短。ITQ 和 SpH 迭代优化的学习过程较为耗时，其中 SpH 就算使用从数据库中部分采样得到的部分子集进行训练也需要耗费大量时间，导致总时间开销较高。与 VDeH 类似，brLSH 也没有显式的学习过程，但其评估 LSH 生成哈希函数的质量时需要进行大量的计算，同时产生极大的时间和空间开销，即使使用从数据库中采样得到子集进行训练也无法有效避免这一问题。总体来看，同时考虑学习和数据转换过程，VDeH 花费的总时间远低于其他方法。

表 4-3　各哈希方法在 T-Drive 数据集上所需的学习时间与数据转换时间（s）

哈希方法	VDeH	ITQ	SpH	brLSH
学习时间	0.17	647.33	3944.11	3081.18
转换时间	228.21	5.09	11.95	3.13
总时间	228.38	652.42	3956.06	3084.31

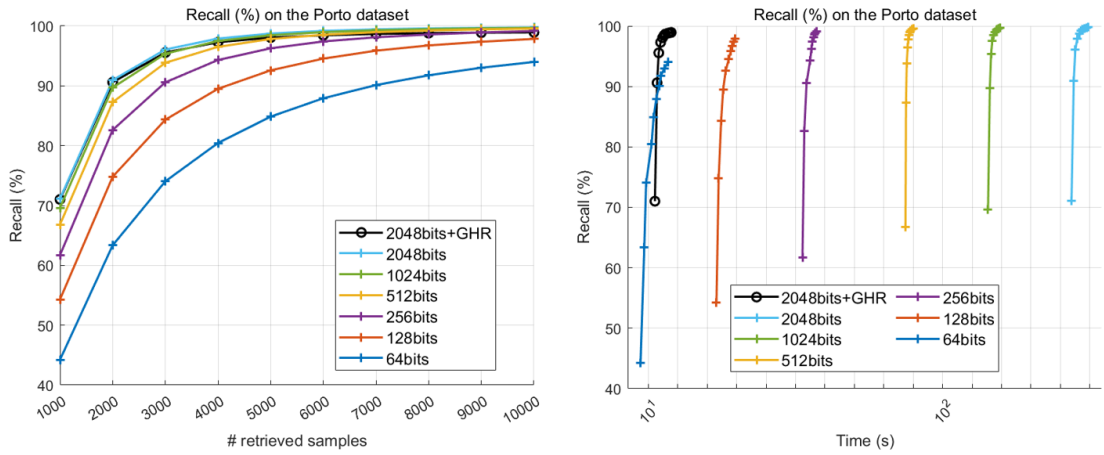
图4-4至图4-6展示了在各数据集上的 1000 最近邻搜索任务中，不同二进制哈希码长度的 VDeH 以及与 GHR 结合的检索方法的召回率。图中纵轴为召回率，子图 (a) 横轴为返回的检索结果数量，子图 (b) 横轴为查询时间。子图 (b) 中曲线召回率与子图 (a) 一一对应，而横坐标即为不同的检索结果数量对应的查询时间。



(a)　　(b)

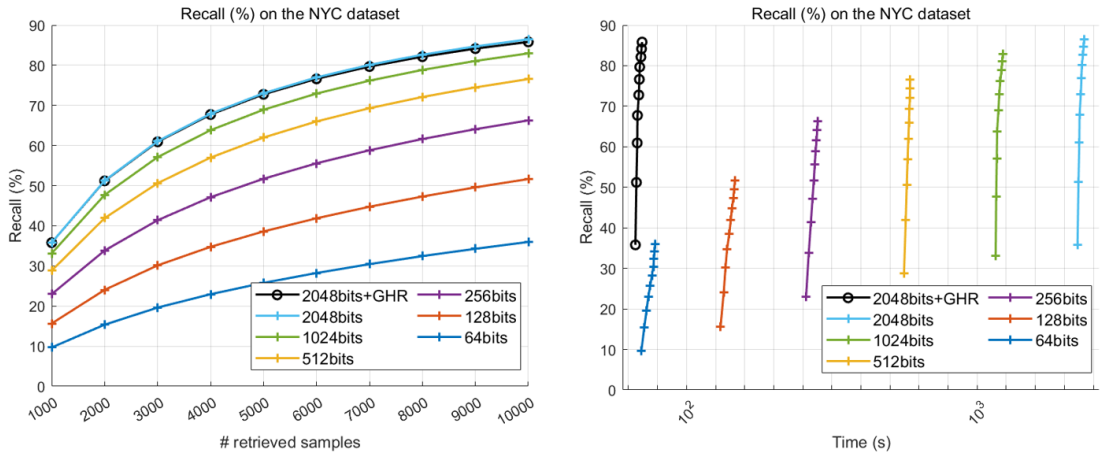
图 4-4　在 T-Drive 数据集上进行 1000 最近邻搜索的召回率

从图中可以看出，VDeH 使用的二进制哈希码长度越长，召回率越高，这与我们在以 mAP 为评价指标的实验中的结果相符。在 T-Drive 和 Porto 数据集上，VDeH 在哈希码长度为 2048 位时的召回率能够接近 100%，即 1000 个真实最近邻数据几乎全部都能被检索返回，而在 NYC 数据集上的召回率则能达到接近



(a) (b)

图 4-5　在 Porto 数据集上进行 1000 最近邻搜索的召回率



(a) (b)

图 4-6　在 NYC 数据集上进行 1000 最近邻搜索的召回率

90%。使用更长的二进制哈希码虽然能够带来性能上的显著提高，但同时也会令查询时间增加，这在图4-4至图4-6的子图 (b) 中有所体现。在相同的检索结果数量下，使用 2048 位哈希码的 VDeH 方法比使用 64 位哈希码的 VDeH 方法具有拥有更高的召回率，但其查询时间增长了近 30 倍。将 VDeH 方法与 GHR 方法结合能够有效解决这一问题。将使用 2048 位哈希码的 VDeH 方法与 GHR 结合以减少考虑的候选最近邻数据量，能使查询时间减少至少一个数量级，达到接近使用 64 位哈希码时的查询时间。而在性能方面，VDeH 与 GHR 结合在 Porto 和 NYC 数据集上几乎没有导致召回率的下降，而在 T-Drive 数据集上也仅有微小的下降。实际上，我们可以调整 GHR 的参数 g 以平衡检索的速度和效果，例如考虑更多的簇（设置更大的 g 值）来将性能的降低缩小到近乎忽略不计，而查

询时间仍保持着较低的水平。从实验结果来看，在最近邻搜索任务中，VDeH 与 GHR 结合的方法能够在保持较高召回率的同时大幅降低查询时间，在大规模数据集上具有更好的检索效率。

4.3 参数分析

对于 VDeH 方法，我们需要构建具有一定数量的沃罗诺伊原胞（Voronoi cell）的沃罗诺伊图。较少的沃罗诺伊原胞数量（对应着普遍较大的沃罗诺伊原胞）会使得沃罗诺伊图的粗糙度增加，查询数据可能的候选最近邻更有可能与查询数据属于同一个沃罗诺伊原胞中，但这些数据相互之间也更难被区分开。而较多的沃罗诺伊原胞数量会让沃罗诺伊图对数据间的相似度刻画更加精细，但同时也更可能导致查询数据的最近邻分布在其它沃罗诺伊原胞中，无法被检索到。因此，我们需要调整沃罗诺伊原胞的数量以获得最佳的检索效果。

图4-7展示了在三个数据集上，使用不同二进制哈希码长度的 VDeH 方法（以 256 位和 512 位为例）对不同沃罗诺伊原胞数量的敏感性，其中横轴为沃罗诺伊原胞数量，纵轴为 mAP 值。可以看出，在 T-Drive 数据集上，当沃罗诺伊原胞数量为 16 时 VDeH 达到最佳性能，而在 Porto 和 NYC 数据集上，沃罗诺伊原胞数量为 256（或是大于 256）时 VDeH 的性能最好，这说明在不同数据集上，由于数据的分布不同，VDeH 应当采用的沃罗诺伊原胞数量往往也是不同的。比较不同二进制哈希码长度下的 VDeH 方法在同一数据集上的表现可以发现，最佳的沃罗诺伊原胞数量并不会随着哈希码长度的变化而发生改变。这意味着在实际应用中，我们可以根据不同数据集的特点，用较短哈希码长度的 VDeH 方法花费较少时间选择出合适的沃罗诺伊原胞数量，而不必担心后续使用更长的二进制哈希码长度会对最佳参数的选择产生影响。

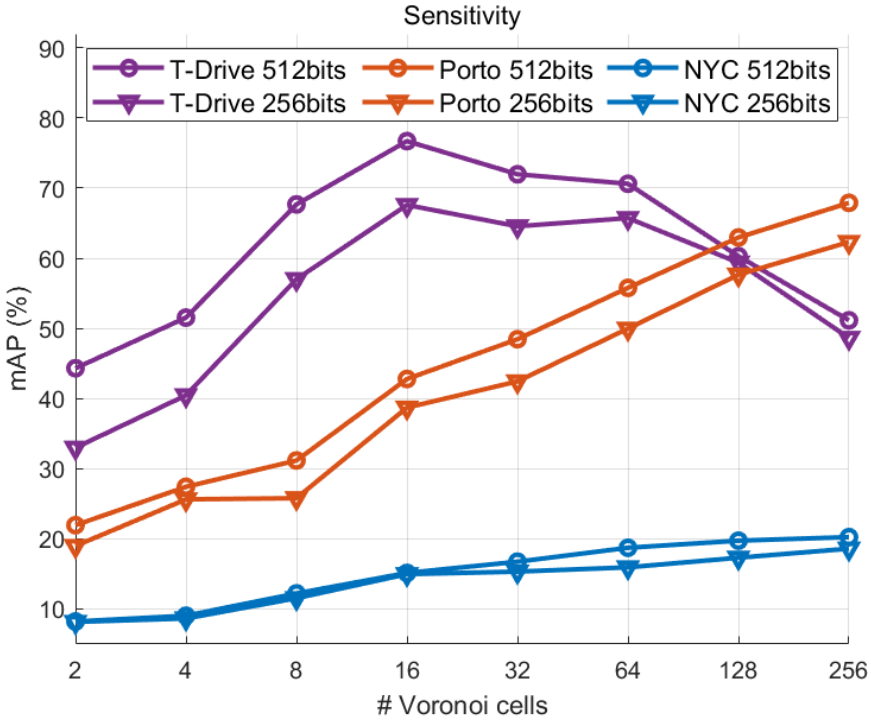


图 4-7　VDeH 对沃罗诺伊原胞数量的敏感性

第五章　总结与展望

在本研究中，我们提出了一种基于 IDK 和沃罗诺伊图的二进制哈希轨迹检索方法 VDeH，通过 IDK 特征映射将轨迹数据映射到 RKHS 中并利用沃罗诺伊图将其转化为二进制哈希码，从而能够实现在大规模轨迹数据集上的快速检索。我们在 T-Drive、Porto 和 NYC 三个真实数据集上进行了实验测试，结果表明 VDeH 在各个数据集上均能取得优异的检索效果，超越了现有的哈希学习方法，且与传统轨迹相似度计算方法相比在大规模数据集上的检索效率具有极大优势。此外，我们还通过与 GHR 方法的结合，进一步提高了 VDeH 在大规模数据集上最近邻数据搜索任务的检索速度。VDeH 对沃罗诺伊原胞数量的敏感性分析表明，在不同数据集上应当采用特定数量的沃罗诺伊原胞以获得最佳的检索效果，而这一参数的选择具有稳定性，不会受到 VDeH 使用的二进制哈希码长度的影响。VDeH 不仅实现了轨迹数据间相似度计算的大幅加速，同时也有效降低了轨迹数据的存储开销，为大规模轨迹数据库的管理和实时应用提供了一种快速高效的解决方案。

展望未来，我们将继续探究 VDeH 与更多类似 GHR 的不同方法进行结合的可能性，以进一步提高 VDeH 在大规模数据集上的检索效率。此外，由于使用轨迹 IDK 特征向量之间的欧氏距离作为基准真实值，在本研究中我们无法与其他加速轨迹相似度计算过程的方法进行比较。我们将在未来的研究中探究如何构建具有合适基准真实值的数据集，以便与更多利用深度学习加速计算轨迹相似度的方法进行公平的比较，从而能够更全面地评估 VDeH 的性能。

参考文献

[1] SU H, LIU S, ZHENG B, et al. A survey of trajectory distance measures and performance evaluation[J]. The VLDB Journal, 2020, 29: 3-32.

[2] YI B K, JAGADISH H V, FALOUTSOS C. Efficient retrieval of similar time sequences under time warping[C]//Proceedings of the 14th International Conference on Data Engineering. 1998: 201-208.

[3] ATEV S, MILLER G, PAPANIKOLOPOULOS N P. Clustering of vehicle trajectories[J]. IEEE Transactions on Intelligent Transportation Systems, 2010, 11(3): 647-657.

[4] YAO D, CONG G, ZHANG C, et al. Computing trajectory similarity in linear time: A generic seed-guided neural metric learning approach[C]//Proceedings of the 35th International Conference on Data Engineering. 2019: 1358-1369.

[5] YAO D, HU H, DU L, et al. Trajgat: A graph-based long-term dependency modeling approach for trajectory similarity computation[C]//Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2022: 2275-2285.

[6] LI X, ZHAO K, CONG G, et al. Deep representation learning for trajectory similarity computation[C]//Proceedings of the 34th International Conference on Data Engineering. 2018: 617-628.

[7] TING K M, XU B C, WASHIO T, et al. Isolation distributional kernel: A new tool for point and group anomaly detections[J]. IEEE Transactions on Knowledge and Data Engineering, 2021, 35(3): 2697-2710.

[8] WANG Z J, ZHU Y, TING K M. Distribution-based trajectory clustering[C]//Proceedings of the 23rd International Conference on Data Mining. 2023: 1379-1384.

[9] WANG J, ZHANG T, SEBE N, et al. A survey on learning to hash[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017, 40(4): 769-790.

[10] CHEN L, ÖZSU M T, ORIA V. Robust and fast similarity search for moving object trajectories[C]//Proceedings of the ACM SIGMOD International Conference on Management of Data. 2005: 491-502.

[11] RANU S, DEEPAK P, TELANG A D, et al. Indexing and matching trajectories under inconsistent sampling rates[C]//Proceedings of the 31st International Conference on Data Engineering. 2015: 999-1010.

[12] CHEN Z, SHEN H T, ZHOU X, et al. Searching trajectories by locations: an efficiency study[C]//Proceedings of the ACM SIGMOD International Conference on Management of Data. 2010: 255-266.

[13] XIE D, LI F, PHILLIPS J M. Distributed trajectory similarity search[J]. Proceedings of the VLDB Endowment, 2017, 10(11): 1478-1489.

[14] INDYK P. Approximate nearest neighbor algorithms for Fréchet distance via product metrics[C]//Proceedings of the 18th Annual Symposium on Computational Geometry. 2002: 102-106.

[15] DRIEMEL A, SILVESTRI F. Locality-sensitive hashing of curves[C]//Proceedings of the 33rd International Symposium on Computational Geometry. 2017: 37-1.

[16] ASTEFANOAEI M, CESARETTI P, KATSIKOULI P, et al. Multi-resolution sketches and locality sensitive hashing for fast trajectory processing[C]//Proceedings of the 26th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems. 2018: 279-288.

[17] WANG Z, LONG C, CONG G, et al. Effective and efficient sports play retrieval with deep representation learning[C]//Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2019: 499-509.

[18] MIKOLOV T, SUTSKEVER I, CHEN K, et al. Distributed representations of words and phrases and their compositionality[J]. Advances in Neural Information Processing Systems, 2013, 26.

[19] MUANDET K, FUKUMIZU K, SRIPERUMBUDUR B, et al. Kernel mean embedding of distributions: A review and beyond[J]. Foundations and Trends® in Machine Learning, 2017, 10(1-2): 1-141.

[20] WEISS Y, TORRALBA A, FERGUS R. Spectral hashing[J]. Advances in Neural Information Processing Systems, 2008, 21.

[21] JEGOU H, DOUZE M, SCHMID C. Product quantization for nearest neighbor search[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2010, 33(1): 117-128.

[22] CHARIKAR M S. Similarity estimation techniques from rounding algorithms[C]//Proceedings of the 34th Annual ACM Symposium on Theory of Computing. 2002: 380-388.

[23] SÁNCHEZ J, PERRONNIN F. High-dimensional signature compression for large-scale image classification[C]//CVPR 2011. 2011: 1665-1672.

[24] JÉGOU H, DOUZE M, SCHMID C, et al. Aggregating local descriptors into a compact image representation[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2010: 3304-3311.

[25] GONG Y, LAZEBNIK S, GORDO A, et al. Iterative quantization: A procrustean approach to learning binary codes for large-scale image retrieval[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2012, 35(12): 2916-2929.

[26] HOTELLING H. Relations between two sets of variates[G]//Breakthroughs in Statistics: Methodology and Distribution. Springer, 1992: 162-190.

[27] HEO J P, LEE Y, HE J, et al. Spherical hashing[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2012: 2957-2964.

[28] CAI D. A revisit of hashing algorithms for approximate nearest neighbor search[J]. IEEE Transactions on Knowledge and Data Engineering, 2019, 33(6): 2337-2348.

[29] LIU H, ZHOU W, WU Z, et al. Refining codes for locality sensitive hashing[J]. IEEE Transactions on Knowledge and Data Engineering, 2023.

[30] ZHOU Z H. Machine learning[M]. Springer Nature, 2021.

[31] TING K M, ZHU Y, ZHOU Z H. Isolation kernel and its effect on SVM[C]//Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2018: 2329-2337.

[32] WILLIAMS C, SEEGER M. Using the Nyström method to speed up kernel machines[J]. Advances in Neural Information Processing Systems, 2000, 13.

[33] RAHIMI A, RECHT B. Random features for large-scale kernel machines[J]. Advances in Neural Information Processing Systems, 2007, 20.

[34] BANDARAGODA T R, TING K M, ALBRECHT D, et al. Isolation-based anomaly detection using nearest-neighbor ensembles[J]. Computational Intelligence, 2018, 34(4): 968-998.

[35] Wikipedia. Voronoi diagram — Wikipedia, The Free Encyclopedia[EB/OL]. 2024. https://en.wikipedia.org/wiki/Voronoi_diagram.

[36] YUAN J, ZHENG Y, XIE X, et al. T-drive: Enhancing driving directions with taxi drivers' intelligence[J]. IEEE Transactions on Knowledge and Data Engineering, 2011, 25(1): 220-232.

[37] MOREIRA-MATIAS L, GAMA J, FERREIRA M, et al. Time-evolving OD matrix estimation using high-speed GPS data streams[J]. Expert Systems with Applications, 2016, 44: 275-288.

[38] DONOVAN B, WORK D. New York City taxi trip data (2010-2013)[EB/OL]. University of Illinois at Urbana-Champaign. 2016. https://doi.org/10.13012/J8PN93H8.

[39] YUAN Y. Habir: Hashing baseline for image retrieval[Z]. https://github.com/willard-yuan/hashing-baseline-for-image-retrieval. 2015.

[40] ZHANG Z, ZOU Q, LIN Y, et al. Improved deep hashing with soft pairwise similarity for multi-label image retrieval[J]. IEEE Transactions on Multimedia, 2019, 22(2): 540-553.

[41] HAN L, LI P, PLAZA A, et al. Hashing for localization (HfL): A baseline for fast localizing objects in a large-scale scene[J]. IEEE Transactions on Geoscience and Remote Sensing, 2021, 60: 1-16.

致　　谢

时光飞逝，光阴荏苒。满怀着对未来的期待初次踏入大学校园的那一刻仿佛还在昨日，而如今本科阶段的终点竟已来到眼前，我也即将迎来人生中的又一个新的起点。回首过去四年，我在南京大学的求学生活充实而美好，有欣喜也有失落，有艰辛也有收获，这段时光将成为我最珍贵的回忆。在此，我要向所有关心我、帮助过我的人致以最诚挚的感谢。

首先要感谢我的导师陈开明教授，他带我走进了科研的大门。加入陈老师的研究组已两年有余，他一直关心着我的学习和生活，尽心尽力地指导我开展研究工作，不厌其烦地解答我的问题，在科研过程中给予了我很多支持和鼓励。陈老师严谨的治学态度、深厚的学术功底和对科研的热情使我受益匪浅，很荣幸能在他的指导下继续深造，他的言传身教将成为我一生的宝贵财富。

其次要感谢研究组里的师兄师姐们，他们在我进组后给予了我很多帮助和关照，让我迅速融入了这个大家庭。感谢许洋师兄、张航师兄对我科研相关问题的指导；感谢祝烨师兄解答我投稿阶段遇到的问题；感谢张凯峰师兄、梁天润师兄帮我解决了实验室机器的问题；感谢商元义师兄、张艺师兄对我的研究和生活上的关心；感谢姜子乐学长教会了首次接触科研的我如何具体开展研究工作。

我还要感谢我的同窗好友邓嘉宏同学和张朝植同学。作为室友，他们为我创造了一个良好的学习和生活环境；作为同学，他们与我一起学习、互相帮助，督促我不断进步；作为朋友，他们让我的大学生活更加丰富多彩。

最后，我尤其要感谢我的家人。感谢我的父母和弟弟在我成长的道路上给予了我最大的支持，是他们的爱让我永远不会感到孤单。他们让我明白，不论我未来选择的道路如何，他们永远都会是我最坚实的后盾。他们的期待和鼓励，就是我不断向前的动力。